



Tools Installation in Ubuntu

SHAKTI Group | CSE Dept | PS-CDISHA - RISE LAB

IIT - Madras

Preview of the Session

1. Hardware Dependencies
2. Vivado Installation
3. Bluespec Compiler Installation
4. RISC-V-GNU-Toolchain Installation
5. Spike Installation
6. Verilator Installation
7. GTKwave and GTKterm

Introduction

A set of Hardware and Software tools are required for Shakti SOC development.

*It is recommended to use a system with Ubuntu 22.04.LTS.

HARDWARE Dependencies:

- Vivado IDE
- Vivado Cable driver.
- Copying Board Files
- Bluespec Compiler
- GTKWave

SOFTWARE Dependencies:

- RISC-V GNU toolchain.
- Spike
- Verilator.
- Gtkterm

+ Installation dependencies

Common Dependencies

The following installation dependencies are required by both Hardware and Software Shakti flow.

Open a terminal and Start Installing!

Command:

```
→ sudo apt-get install autoconf automake autotools-dev curl make libmpc-dev libmpfr-dev  
libgmp-dev bc gawk build-essential bison flex texinfo gperf libtool patchutils zlib1g-dev  
pkg-config
```

Vivado Installation

A set of Vivado dependencies which are used for connecting FPGA boards to Vivado and programming the FPGA boards needs to be installed before starting vivado installation.

Command:

→ **sudo apt-get install libusb-1.0-0-dev libusb-0.1-4 libftdi1 libftdi1-2 libncurses5**

After installing the above dependencies, proceed on to download the Vivado software from the Link provided.

https://www.xilinx.com/member/forms/download/xef-vivado.html?filename=Xilinx_Vivado_SDK_2018.3_1207_2324.tar.gz

Note: *

A login is required to download the tar file in the AMD website.

Higher versions also can be installed.

Vivado Installation

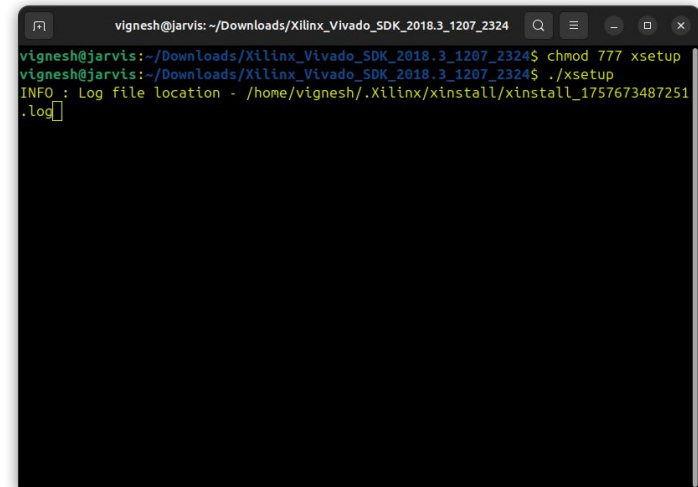
After downloading, go to the file location and untar the file "<filename>.tar.xz"

→ **tar xf Xilinx_Vivado_SDK_2018.3_1207_2324.tar.gz**

Go inside the Untarred folder and update file permission (executable) of the file "xsetup" by using the command

→ **chmod 777 xsetup**

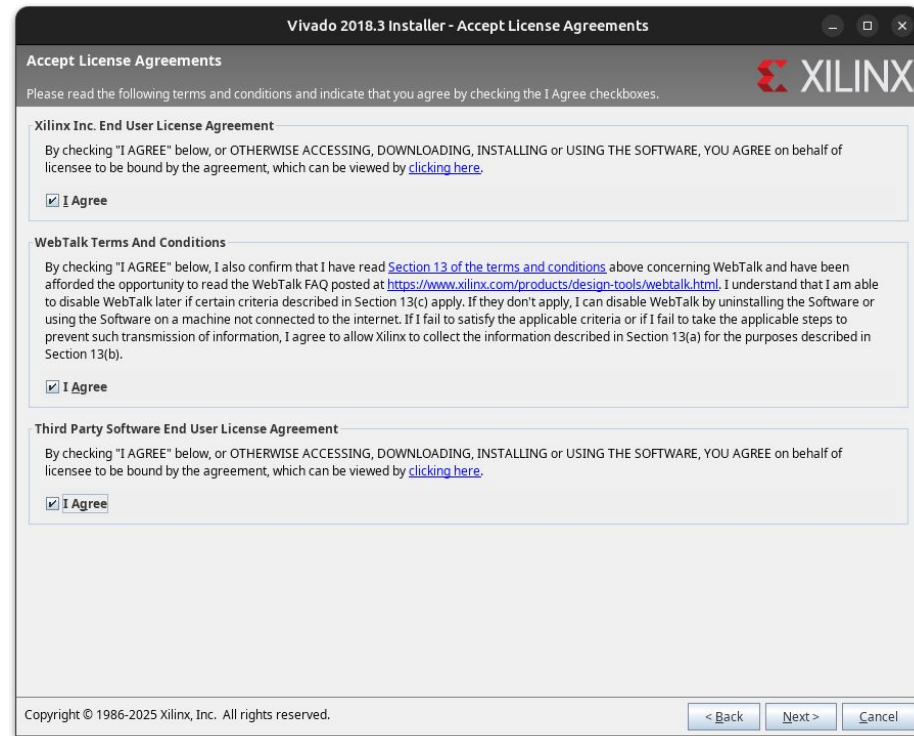
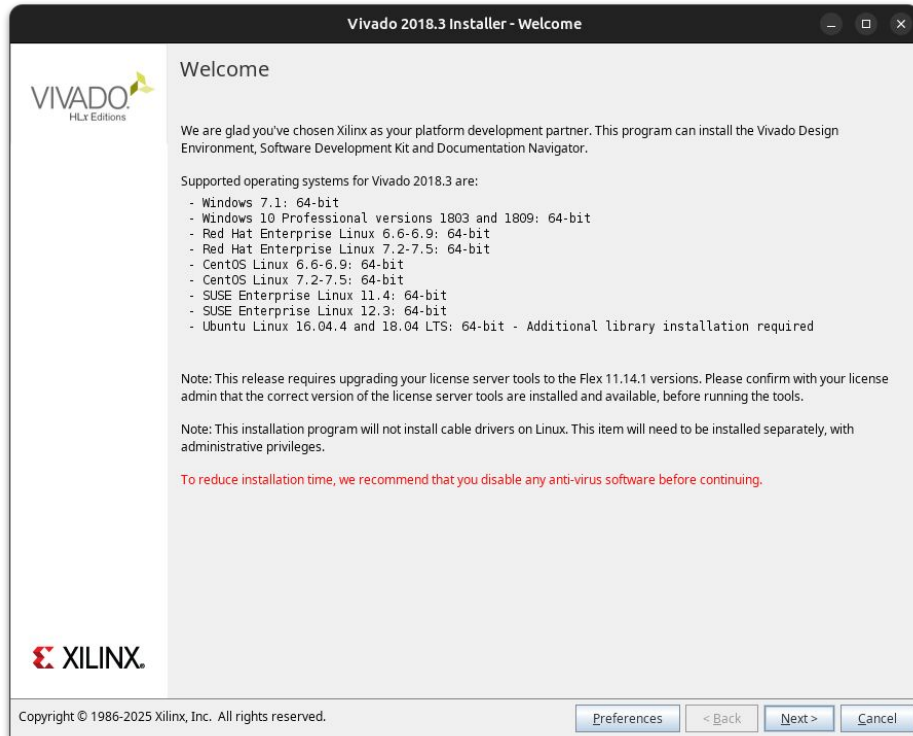
Run the code "./xsetup" (./xsetup) in the terminal"

A terminal window with a dark background and light green text. The window title is "vignesh@jarvis: ~/Downloads/Xilinx_Vivado_SDK_2018.3_1207_2324". The command prompt shows "vignesh@jarvis:~/Downloads/Xilinx_Vivado_SDK_2018.3_1207_2324\$ chmod 777 xsetup". The next line shows "vignesh@jarvis:~/Downloads/Xilinx_Vivado_SDK_2018.3_1207_2324\$./xsetup". The output shows "INFO : Log file location - /home/vignesh/.Xilinx/xinstall/xinstall_1757673487251.log".

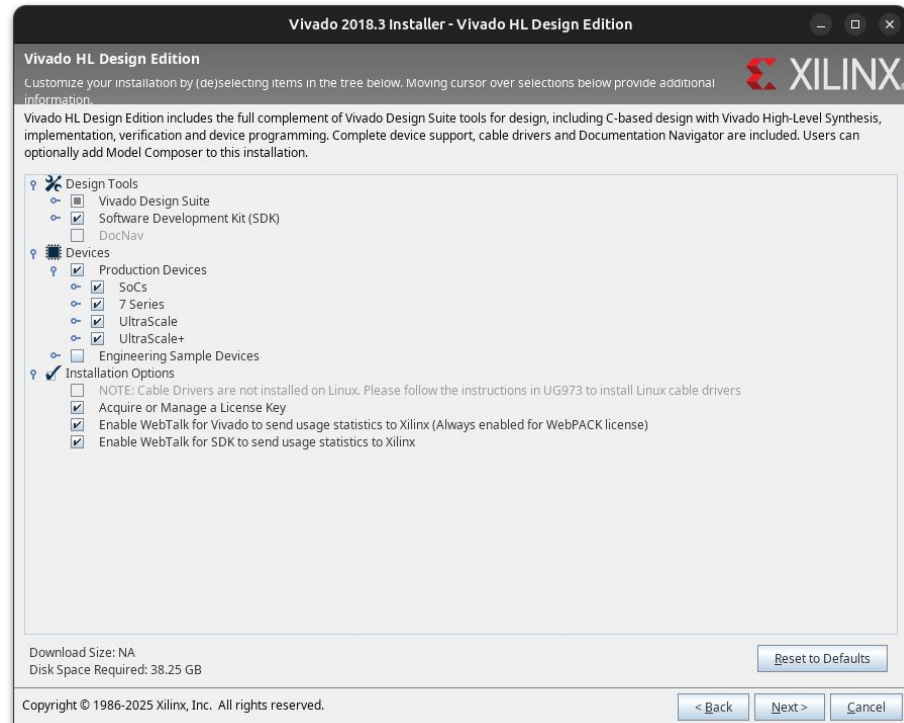
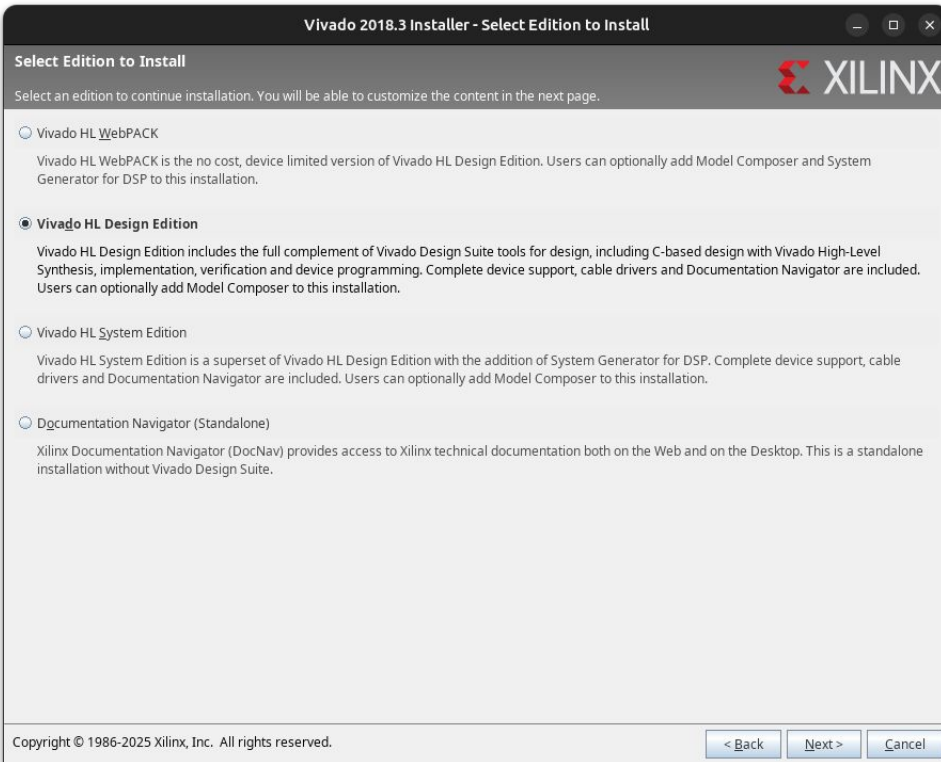
```
vignesh@jarvis: ~/Downloads/Xilinx_Vivado_SDK_2018.3_1207_2324
vignesh@jarvis:~/Downloads/Xilinx_Vivado_SDK_2018.3_1207_2324$ chmod 777 xsetup
vignesh@jarvis:~/Downloads/Xilinx_Vivado_SDK_2018.3_1207_2324$ ./xsetup
INFO : Log file location - /home/vignesh/.Xilinx/xinstall/xinstall_1757673487251.log
```



Vivado Installation flow



Vivado Installation flow



Vivado Installation flow

Vivado 2018.3 Installer - Select Destination Directory

Select Destination Directory

Choose installation options such as location and shortcuts.

Installation Options

Select the installation directory

/scratch/dummy

Installation location(s)

/scratch/dummy/Vivado/2018.3

/scratch/dummy/SDK/2018.3

Disk Space Required

Download Size: NA

Disk Space Required: 38.25 GB

Disk Space Available: 1824.98 GB

Select shortcut and file association options

☐ Create program group entries

Xilinx Design Tools

☒ Create desktop shortcuts



Copyright © 1986-2025 Xilinx, Inc. All rights reserved.

< Back

Next >

Cancel

Vivado 2018.3 Installer - Installation Summary



Installation Summary

Edition: Vivado HL Design Edition

Devices

- Production Devices (SoCs, 7 Series, UltraScale, UltraScale+)

Design Tools

- Vivado Design Suite (Vivado)
- Software Development Kit (SDK) (SDK Core Tools, Compiler Tool Chains)

Installation Options

- Enable WebTalk for SDK to send usage statistics to Xilinx
- Enable WebTalk for Vivado to send usage statistics to Xilinx (Always enabled for WebPACK license)
- Acquire or Manage a License Key

Installation location

- /scratch/dummy/Vivado/2018.3
- /scratch/dummy/SDK/2018.3

Disk Space Required

- Download Size: NA
- Disk Space Required: 38.25 GB



Copyright © 1986-2025 Xilinx, Inc. All rights reserved.

Preferences

< Back

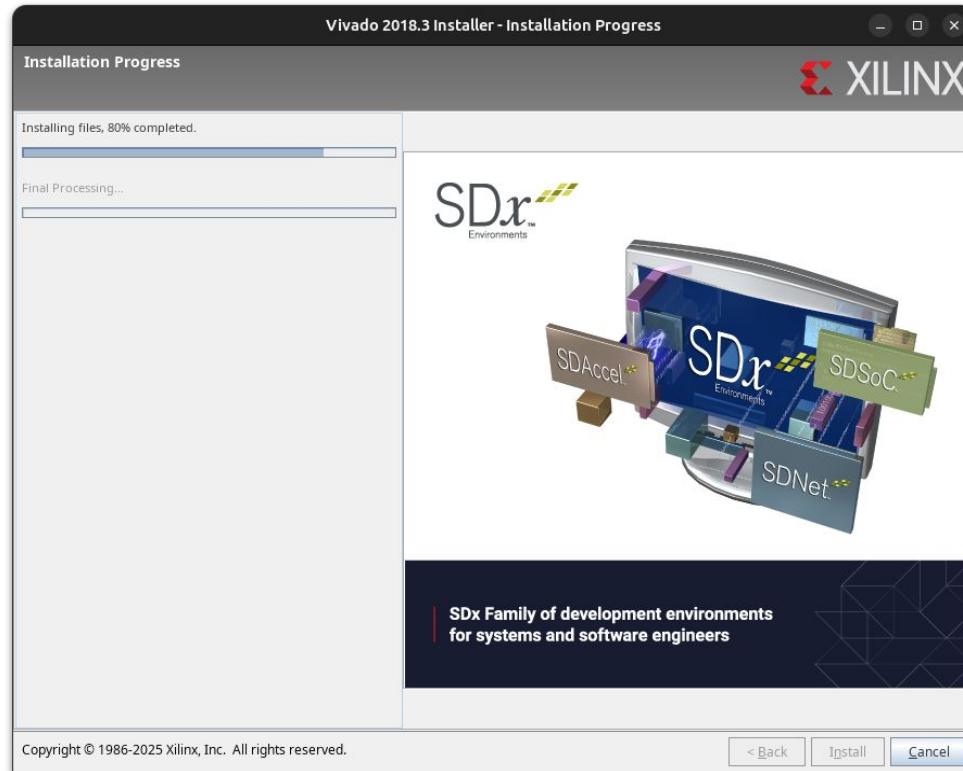
Install

Cancel



SHAKTI

Vivado Installation flow

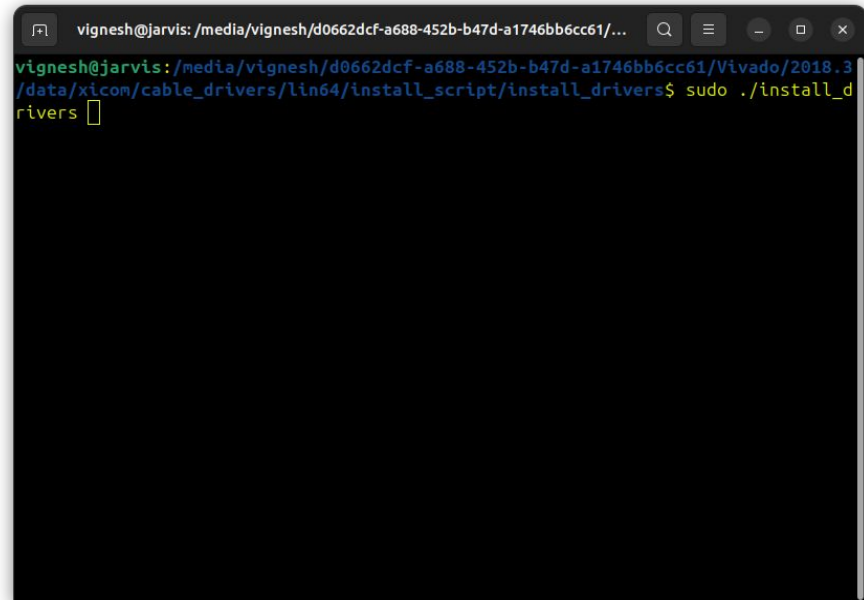


Post Installation

After installation, do the following.

1. Go to the folder “[installed folder]/vivado/Vivado/2018.3/data/xicom/cable_drivers/lin64/install_script/install_drivers/”
2. run “**sudo ./install_drivers**”

*These drivers are not available pre-installed in the Vivado, so it needs to be installed before connecting the board.

A terminal window with a dark background. The title bar shows the file path: /media/vignesh/d0662dcf-a688-452b-b47d-a1746bb6cc61/... The terminal text shows the user 'vignesh@jarvis' at the prompt, followed by the command 'sudo ./install_drivers' which has been executed, as indicated by the green prompt character and the absence of a cursor.

```
vignesh@jarvis: /media/vignesh/d0662dcf-a688-452b-b47d-a1746bb6cc61/...  
vignesh@jarvis: /media/vignesh/d0662dcf-a688-452b-b47d-a1746bb6cc61/Vivado/2018.3  
/data/xicom/cable_drivers/lin64/install_script/install_drivers$ sudo ./install_d  
rivers
```



Adding board files

Board files can be added on cloning the board files from diligent github page.

a. Clone the repo.

→ **git clone** <https://github.com/Digilent/vivado-boards.git>

b. Go to the folder `./Digilent/vivado-boards/tree/master/new/board_files` and copy them.

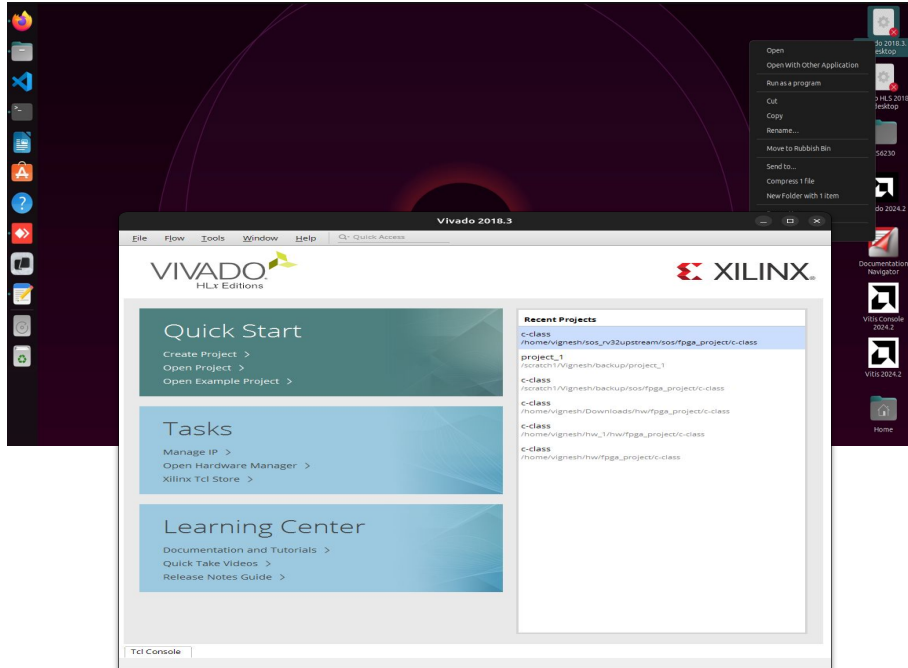
c. Paste the files to `[vivado installed path]/Vivado/2018.3/data/boards/board_files`.

Invoking Vivado application

Vivado application can be invoked in two following ways.

1. Through icon in home:

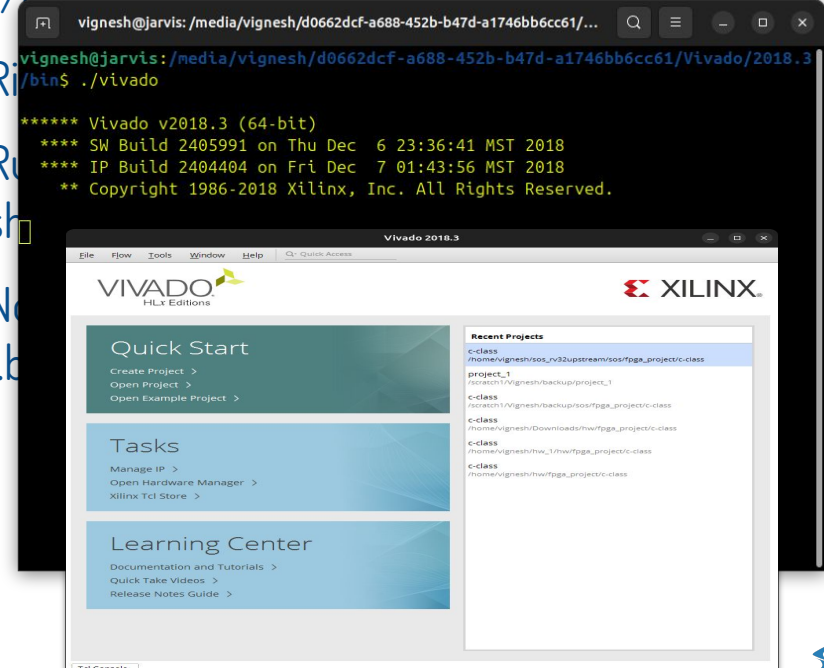
1. Go to the Desktop and find the vivado icon



2. Through Terminal

Go to following path

“/path/install/vivado/Vivado/2018.3/”



Bluespec Compiler Installation

Installation of Bluespec compiler is necessary as almost everything in and around **Shakti** is on Bluespec.

1. Install these specific dependencies and clone the repository:

→ **sudo apt-get install ghc libghc-regex-compat-dev libghc-syb-dev libghc-old-time-dev libghc-split-dev tcl-dev**

2. Clone the repo “ **git clone --recursive <https://github.com/B-Lang-org/bsc>”**

3. Go into bsc , checkout the git to 2025.01.1 branch and install from source:

→ **cd bsc**

→ **git checkout 2025.01.1**

→ **sudo make install-src**

Bluespec Compiler Installation

Add the bluespec to the PATH in Linux by adding it to **.bashrc**

→ **echo 'export PATH=/home/test/bsc/inst/bin:\$PATH' >> ~/.bashrc**

Bluespec compiler can be invoked easily with a small command:

→ **bsc**

```
test@shakti:~$ bsc
Usage:
bsc -help                to get help
bsc [flags] file.bsv     to partially compile a Bluespec file
bsc [flags] -verilog -g mod file.bsv to compile a module to Verilog
bsc [flags] -verilog -g mod -u file.bsv to recursively compile modules to Verilog
bsc [flags] -verilog -e topmodule    to link Verilog into a simulation model
bsc [flags] -sim -g mod file.bsv     to compile to a Bluesim object
bsc [flags] -sim -g mod -u file.bsv  to recursively compile to Bluesim objects
bsc [flags] -sim -e topmodule        to link objects into a Bluesim binary
bsc [flags] -systemc -e topmodule    to link objects into a SystemC model
```



RISC-V GNU Toolchain Installation

Install these specific dependencies and clone the repository:

→ **sudo apt-get install python3 python3-pip libexpat-dev ninja-build git cmake libglib2.0-dev libslirp-dev**

→ **git clone <https://github.com/riscv/riscv-gnu-toolchain>**

After cloning go into the folder , update the submodule, change version of gcc to 13.2 ,configure and make. These commands will configure ,make and install the toolchain. This will contain both 32-bit and 64-bit toolchain

→ **git checkout releases/gcc-13.2.0**

→ **cd riscv-gnu-toolchain**

→ **cd ..**

→ **git submodule update --init --recursive**

→ **mkdir build_rv**

→ **sudo rm -rf gcc**

→ **./configure**

→ **git clone <https://gcc.gnu.org/git/gcc.git>**

--prefix=/path/to/toolchain/build_rv

→ **cd gcc**

--enable-multilib

→ **sudo make -j \$(nproc)**

RISC-V GNU Toolchain Installation

Add the path to .bashrc

→ **echo 'export PATH=[Path to risc-v toolchain]/riscv-gnu-toolchain/build_rv/bin:\$PATH' >> ~/.bashrc**

To verify try these commands:

→ **which riscv64-unknown-elf-gcc**

→ **riscv64-unknown-elf-gcc --print-multi-lib**

```
vignesh@jarvis:~$ which riscv64-unknown-elf-gcc
/media/vignesh/d0662dcf-a688-452b-b47d-a1746bb6cc61/riscv-gnu-toolchain/build_rv/bin/riscv64-unknown-elf-gcc
```

```
vignesh@jarvis:~$ riscv64-unknown-elf-gcc --print-multi-lib
.;
rv32i/ilp32;@march=rv32i@mabi=ilp32
rv32im/ilp32;@march=rv32im@mabi=ilp32
rv32iac/ilp32;@march=rv32iac@mabi=ilp32
rv32imac/ilp32;@march=rv32imac@mabi=ilp32
rv32imafc/ilp32f;@march=rv32imafc@mabi=ilp32f
rv64imac/lp64;@march=rv64imac@mabi=lp64
rv64imafdc/lp64d;@march=rv64imafdc@mabi=lp64d
```



Spike Installation

Install these specific dependencies and clone the git from this repository :

→ **sudo apt-get install device-tree-compiler libboost-regex-dev libboost-system-dev**

→ **git clone <https://github.com/riscv-software-src/riscv-isa-sim.git>**

Go inside the cloned directory and checkout to the specified branch:

→ **cd riscv-isa-sim**

→ **git checkout d39cbb3ad27b2de882260c0a22f1a59efebb151**

Spike Installation

Create a build directory , configure it and make:

→ **mkdir build**

→ **cd build**

→ **export RISCV=/path/to/install/spike**

→ **./configure --prefix=\$RISCV**

→ **sudo make -j \${nproc}**

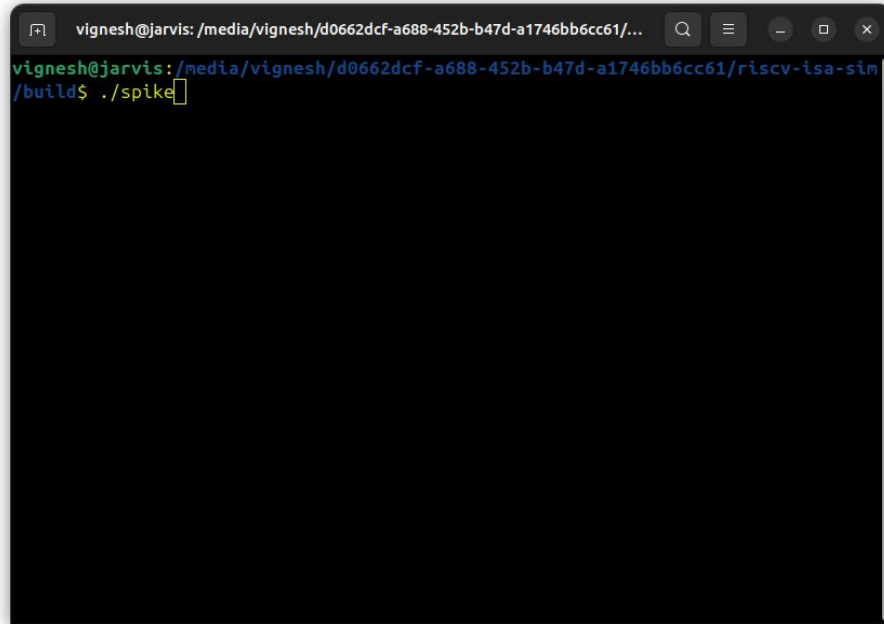
→ **sudo make install**



Spike Installation

After Installation, Spike can be invoked with this command:

→ **./spike**



```
vignesh@jarvis: /media/vignesh/d0662dcf-a688-452b-b47d-a1746bb6cc61/...  
vignesh@jarvis: /media/vignesh/d0662dcf-a688-452b-b47d-a1746bb6cc61/riscv-isa-sim  
/build$ ./spike
```



Verilator Installation

Install the specific dependencies and clone the git from the specified directory:

→ **sudo apt-get install perl g++ ccache libgoogle-perftools-dev numactl perl-doc libfl2 libfl-dev zlib1g help2man**

→ **git clone <https://github.com/verilator/verilator>**

Go into the cloned directory and checkout to the specified branch:

→ **cd verilator**

→ **git checkout v5.024**

Verilator Installation

Install Verilator through these commands:

→ **unset VERILATOR_ROOT**

→ **autoconf**

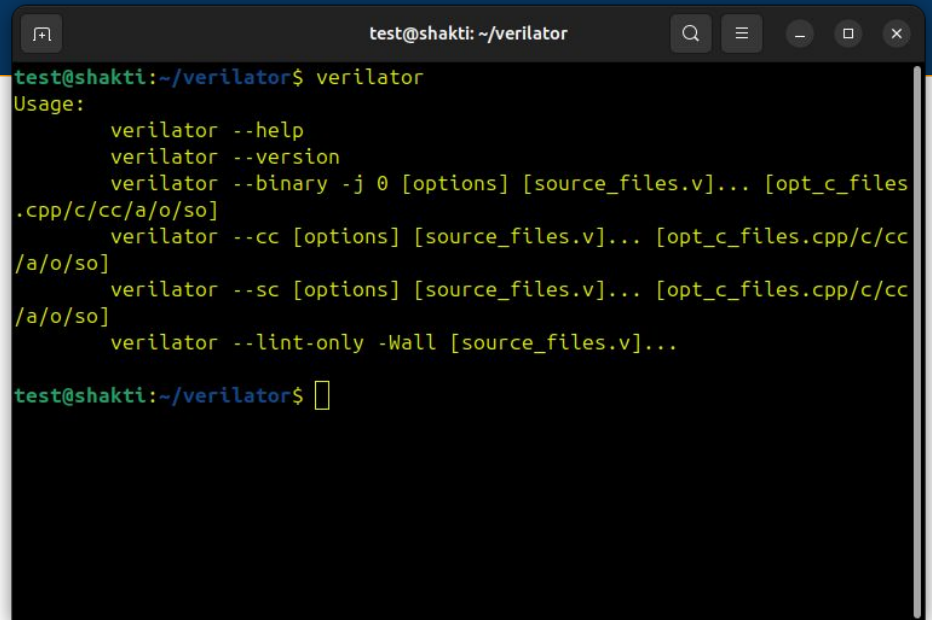
→ **./configure**

→ **make -j `nproc`**

→ **sudo make install**

After Installation, we can check if verilator is there by giving,

→ **verilator**

A terminal window titled 'test@shakti: ~/verilator' showing the output of the 'verilator' command. The output displays the usage of the verilator tool, including options for help, version, binary generation, compilation, simulation, and linting.

```
test@shakti:~/verilator$ verilator
Usage:
  verilator --help
  verilator --version
  verilator --binary -j 0 [options] [source_files.v]... [opt_c_files
.cpp/c/cc/a/o/so]
  verilator --cc [options] [source_files.v]... [opt_c_files.cpp/c/cc
/a/o/so]
  verilator --sc [options] [source_files.v]... [opt_c_files.cpp/c/cc
/a/o/so]
  verilator --lint-only -Wall [source_files.v]...
```

GTKterm and GTKwave

GTKwave → simulation software for checking the waveforms for the specific code.

GTKterm → emulator used for getting outputs from serial ports.

Installation of GTKwave and GTKterm is pretty simple, and are done through these commands:

→ **sudo apt install gtkterm -y**

→ **sudo apt install gtkwave -y**

GTKterm and GTKwave can be used very easily through these commands:

→ **sudo gtkterm**

→ **sudo gtkwave**



Thank you

